

AGENTIC OS

Estudiante: Mario Alberto Alvarado Giler
Paralelo: 1

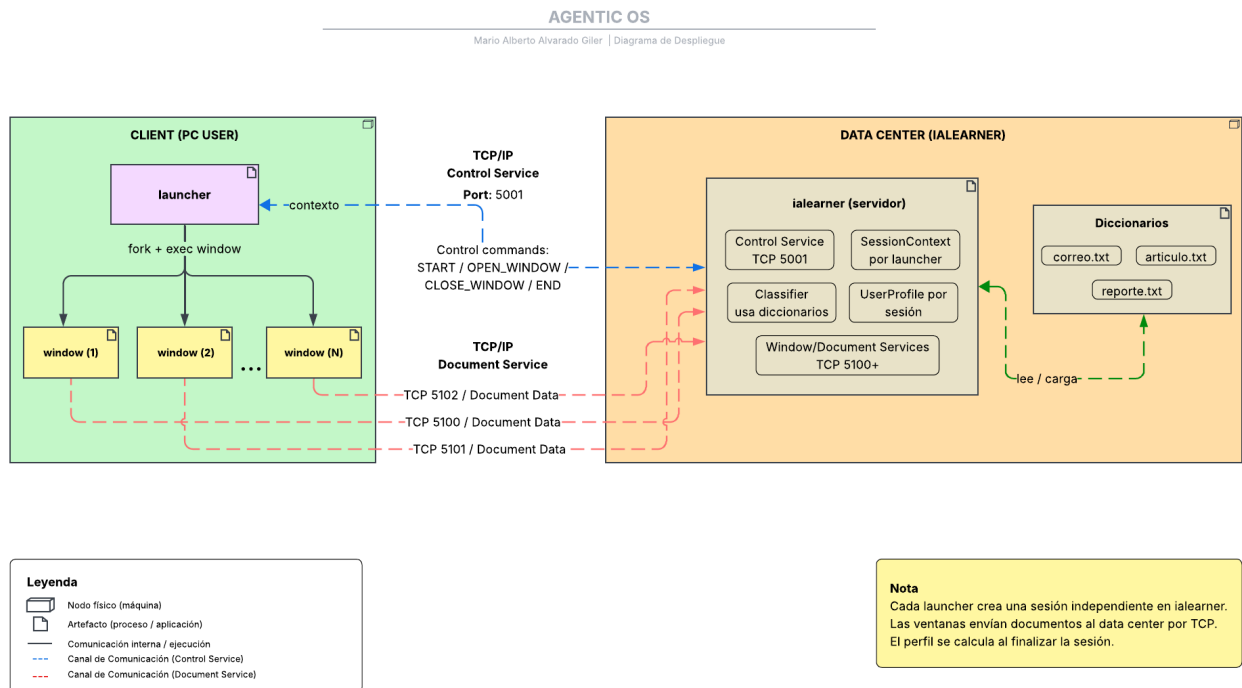


Figura 1. Diagrama de Despliegue del proyecto Agentic OS

El sistema presente se despliega siguiendo una arquitectura cliente-servidor como se muestra en la Figura 1. El proceso **ialearner** representa el nodo central o data center del sistema. Este proceso se ejecuta primero y permanece activo esperando conexiones de launchers. Su responsabilidad principal es recibir documentos generados por las ventanas, clasificarlos usando diccionarios locales y construir un perfil de usuario por cada sesión activa.

El data center expone un servicio de control mediante **sockets TCP** en el puerto **5001**. Por este canal se comunican los procesos launcher con ialearnner. El launcher envía comandos como iniciar sesión, solicitar una nueva ventana, cerrar una ventana o finalizar la sesión. Cuando un launcher solicita crear una ventana, ialearnner asigna un puerto disponible a partir del rango 5100, 5101, 5102, ... Esta decisión evita conflictos cuando existen varios launchers conectados al mismo tiempo, ya que el servidor central administra globalmente los puertos de las ventanas.

Cada computadora de usuario ejecuta su propio proceso **launcher**. El launcher actúa como administrador local de ventanas, permite crear ventanas, cerrar ventanas, mostrar procesos y finalizar la sesión. Las ventanas se ejecutan como **procesos** separados llamados **window**. Cada ventana se conecta por **TCP** al puerto asignado por ialearnner y envía los caracteres escritos por el usuario. Cuando la ventana se cierra, el servidor procesa el documento recibido y lo clasifica.

La clasificación se realiza en el data center usando tres diccionarios: **correo.txt**, **articulo.txt** y **reporte.txt**. Según las palabras encontradas en el documento, el sistema determina si corresponde a correo electrónico, artículo o reporte. Luego, la sesión del usuario actualiza su perfil interno. Cada launcher tiene una sesión independiente, por lo que dos usuarios conectados al mismo data center no mezclan sus perfiles ni sus ventanas.

Para manejar concurrencia, **ialearner** utiliza **hilos**. Cada launcher conectado se atiende en una sesión propia, y las ventanas también son atendidas concurrentemente. Como varias partes del sistema pueden acceder a recursos compartidos, se usan **mutex** para proteger **secciones críticas**, como el perfil de usuario de cada sesión y la asignación global de puertos. Esto evita condiciones de carrera cuando varios clientes o ventanas trabajan al mismo tiempo.

El sistema también contempla fallos de comunicación. Si **ialearner** se apaga bruscamente o se pierde la conexión con el data center, el launcher detecta que el servicio ya no está disponible, cierra las ventanas locales que dependen de él y finaliza. Esta medida evita dejar procesos huérfanos sin conexión al servicio central.

En una ejecución local, todos los procesos pueden correr en la misma máquina usando **127.0.0.1**. Sin embargo, conceptualmente el despliegue representa un escenario distribuido: **ialearner** puede estar en una máquina servidor y varios launchers pueden ejecutarse en computadoras cliente, conectándose al mismo data center mediante sockets TCP.